

Bezpieczeństwo przesyłania danych, Zadanie 6

Piotr Gugnowski, wydział EiTI, nr indeksu 320690

Czerwiec 2026

Opis sytuacji

Skaner zainstalowany przy jednostce master sieci Modbus RTU przechwycił ramkę FC 0x10 (Write Multiple Registers) z nieczytelnym 128-bitowym kodem dostępu. Znana jest jedynie suma kontrolna **CRC kodu** = 0xA77C, wyznaczona zgodnie ze standardową specyfikacją Modbus CRC-16.

Pole	Bajty	Wartość	Opis
Adres slave	1	0x0A	adres urządzenia docelowego
Kod funkcji	1	0x10	Write Multiple Registers
Adres pierwszego rej.	2	0x0000	rejestr startowy kodu
Adres ostatniego rej.	2	0x0008	rejestry 0 do 7: kod, rejestr 8: CRC kodu
Liczba bajtów danych	1	0x12	18 B = 16 B kodu + 2 B CRC kodu
Rejestr 0 (kod)	2	0x????	bajty kodu 1 i 2
Rejestr 1 (kod)	2	0x????	bajty kodu 3 i 4
Rejestr 2 (kod)	2	0x????	bajty kodu 5 i 6
Rejestr 3 (kod)	2	0x????	bajty kodu 7 i 8
Rejestr 4 (kod)	2	0x????	bajty kodu 9 i 10
Rejestr 5 (kod)	2	0x????	bajty kodu 11 i 12
Rejestr 6 (kod)	2	0x????	bajty kodu 13 i 14
Rejestr 7 (kod)	2	0x????	bajty kodu 15 i 16
Rejestr 8 (CRC kodu)	2	0xA77C	CRC-16/Modbus z 16 B kodu
CRC ramki Modbus	2	0x????	CRC-16 całej ramki

Tabela 1: Przechwycona ramka Modbus RTU. Pole danych: 8 rejestrów kodu (16 B) + 1 rejestr CRC kodu (2 B) = 18 B.

Zadanie 6a: Odtworzenie kodu dostępu (4 pkt)

Treść zadania

Odtworzyć kod dostępu do zamka składu broni.

Analiza

Jedyna znana informacja o kodzie to jego 16-bitowa suma kontrolna CRC-16/Modbus = 0xA77C. Warunku $CRC16(k) = 0xA77C$ spełnia nie jeden, lecz 2^{112} różnych 128-bitowych ciągów (do-

wód w Zadaniu 6b). Nie jest zatem możliwe **jednoznaczne** odtworzenie kodu, można jedynie wygenerować *jeden z możliwych* kodów pasujących do danego CRC.

Metoda wyznaczenia przykładowego kodu

Korzystamy z bijekcyjności odwzorowania $(b_{15}, b_{16}) \mapsto \text{CRC16}(\text{kod})$ przy ustalonych pierwszych 14 bajtach (wykazanej w Zadaniu 6b).

1. Wybieramy dowolnie **pierwsze** 14 bajtów kodu (bajty 1–14, lewa strona). Dla uproszczenia przyjmujemy zera:

$$b_1 = b_2 = \dots = b_{14} = 0x00$$

Dowolny inny wybór tych 14 bajtów również byłby poprawny i dałby inną parę (b_{15}, b_{16}) spełniającą warunek CRC.

2. Obliczamy stan rejestru CRC po przetworzeniu tych 14 bajtów. CRC jest algorytmem **sekwencyjnym**: stan po bajcie i zależy wyłącznie od bajtów $b_1, \dots, b_i$ i nie wymaga znajomości kolejnych bajtów. Bajty b_{15}, b_{16} jeszcze nie zostały przetworzone, więc stan CRC_state_{14} liczymy niezależnie od nich (inicjalizacja $0xFFFF$, każdy bajt $0x00$ daje XOR bez zmiany, jedynie iteracje LFSR modyfikują stan):

Nr bajtu	Bajt	Stan CRC po bajcie
init		0xFFFF
1	0x00	0x40BF
2	0x00	0xB001
3	0x00	0xC071
4	0x00	0x2400
5	0x00	0x0024
6	0x00	0x1B00
7	0x00	0x001B
8	0x00	0x0B40
9	0x00	0xF00A
10	0x00	0x0770
11	0x00	0xE406
12	0x00	0x0264
13	0x00	0xEB03
14	0x00	0x01AB

$$\text{CRC_state}_{14} = 0x01AB$$

3. Bajtów 15 i 16 **nie przepuszczamy przez LFSR jako niewiadomych**, bo każda iteracja LFSR zależy od aktualnego LSB rejestru, który jest wyznaczany dopiero po XOR z bajtem wejściowym. Zamiast tego **przeszukujemy** wszystkie $256 \times 256 = 65\,536$ możliwych kombinacji (b_{15}, b_{16}) : dla każdej pary obliczamy CRC ze stanu $0x01AB$ i sprawdzamy, czy wynik to $0xA77C$:

$$\text{crc16_update}(\text{crc16_update}(0x01AB, b_{15}), b_{16}) \stackrel{?}{=} 0xA77C$$

4. Wynik: $b_{15} = 0xF9$, $b_{16} = 0x0A$. Przebieg obliczeń:

$$0x01AB \xrightarrow{b_{15}=0xF9} 0xFD80 \xrightarrow{b_{16}=0x0A} 0xA77C \checkmark$$

Przykładowy kod dostępu

Rejestr	Bajty (hex)	Opis
0	00 00	bajty 1 i 2
1	00 00	bajty 3 i 4
2	00 00	bajty 5 i 6
3	00 00	bajty 7 i 8
4	00 00	bajty 9 i 10
5	00 00	bajty 11 i 12
6	00 00	bajty 13 i 14
7	F9 0A	bajty 15 i 16 (wyznaczone z CRC)
Kod (hex): 0000000000000000000000000000F90A		
Weryfikacja: CRC16(kod) = 0xA77C		✓

Tabela 2: Jeden z 2^{112} możliwych kodów dostępu pasujących do CRC = 0xA77C.

Kompletna odtworzona ramka

Mając przykładowy kod, można uzupełnić całą ramkę Modbus. CRC ramki obliczane jest ze wszystkich bajtów od adresu slave do CRC kodu włącznie (patrz Dodatek B).

Pole	Bajty (hex)	Obliczone CRC po tym polu
Adres slave 0x0A	0A	0x473F
Kod funkcji 0x10	10	0xDC06
Adres reg. 0x0000	00 00	0x3902
Liczba reg. 0x0008	00 08	0xB4C0
Liczba bajtów 0x12	12	0x5D34
Dane (14 bajtów 0x00)	00×14	0x228B
Dane bajt 15: 0xF9	F9	0xC8A8
Dane bajt 16: 0x0A	0A	0xB949
CRC kodu 0xA77C	A7 7C	0xF38D
CRC ramki	8D F3	(bajt niski, bajt wysoki)

Tabela 3: Krok po kroku obliczanie CRC ramki. Wartość CRC = 0xF38D; w ramce: bajt niski 0x8D jako pierwszy, bajt wysoki 0xF3 jako drugi.

Kompletna ramka (27 bajtów):

0A 10 00 00 00 08 12 00 00 00 00 00 00 00 00 00 00 00 00 00 00 F9 0A A7 7C 8D F3

Zadanie 6b: Liczba możliwych kodów dostępu (4 pkt)

Treść zadania

Ile jest możliwych kodów dostępu? Odpowiedź udowodnić.

Dowód

Twierdzenie. Liczba 128-bitowych kodów dostępu $k \in \{0, 1\}^{128}$ spełniających $\text{CRC16}(k) = 0xA77C$ wynosi dokładnie 2^{112} .

Dowód.

Podzielmy 16-bajtowy kod na dwie części:

$$k = (\underbrace{b_1, \dots, b_{14}}_{\text{prefiks, 112 bitów}} , \underbrace{b_{15}, b_{16}}_{\text{sufiks, 16 bitów}})$$

Kluczowy lemat: dla każdego wyboru prefiksu $(b_1, \dots, b_{14})$ istnieje **dokładnie jeden** sufiks (b_{15}, b_{16}) taki, że $\text{CRC16}(k) = 0xA77C$.

Jeśli lemat jest prawdziwy, zliczanie jest natychmiastowe:

$$\underbrace{2^{112}}_{\text{możliwych prefiksów}} \times \underbrace{1}_{\text{sufiks na prefiks}} = 2^{112}$$

Dowód lematu.

CRC jest algorytmem sekwencyjnym: jedyną “pamięcią” po przetworzeniu prefiksu jest bieżący stan rejestru $R \in \{0, \dots, 65535\}$ (liczba 16-bitowa). Bez względu na to, jakie bajty tworzyły prefiks, dwa prefiksy prowadzące do tego samego stanu R są od tej chwili **nierozróżnialne** dla algorytmu.

Lemat jest zatem równoważny stwierdzeniu:

Dla każdego stanu startowego R , odwzorowanie $(b_{15}, b_{16}) \mapsto \text{CRC16_continue}(R, b_{15}, b_{16})$ jest bijekcją (każdy wynik CRC pojawia się dokładnie raz).

Stany startowe to liczby 16-bitowe, jest ich dokładnie 65 536. Dla każdego z nich sprawdzamy wszystkie 65 536 par (b_{15}, b_{16}) i weryfikujemy, że wyniki są parami różne (każdy wynik jest jedną z 65 536 możliwych wartości CRC, więc para różnych wejść dająca ten sam wynik oznaczałaby kolizję). Łączna liczba obliczeń: $65\,536^2 \approx 4,3 \times 10^9$. Każde obliczenie to dwa kroki LFSR z tablicą lookup (jedno odczytanie z pamięci i jeden XOR na krok), co umożliwia wykonanie całości w czasie poniżej 4 sekund. Jest to **skończone, wyczerpujące sprawdzenie** wykonane programem `verify_bijection.cpp` (kod źródłowy w pliku `verify_bijection.cpp`, wymagania i instrukcja budowania w pliku `README.md`, kompilacja na macOS: `just build`).

Wynik działania programu `verify_bijection`:

```
Sprawdzono stanów: 65536
Stanów z kolizją: 0
Bijekcja dla wszystkich stanów: TAK
```

Dla każdego prefiksu istnieje zatem dokładnie jeden sufiks. Prefiksów jest 2^{112} , co kończy dowód:

$$\boxed{\text{Liczba możliwych kodów} = 2^{112}}$$

Zadanie 6c: Informacje do jednoznacznego wyznaczenia kodu (2 pkt)

Treść zadania

Proszę o komentarz, jakie informacje należałoby zdobyć, aby kod został wyznaczony w sposób jednoznaczny.

Odpowiedź

Dysponujemy jedynie 16-bitowym CRC kodu. Kod ma 128 bitów, więc brakuje $128 - 16 = 112$ bitów informacji do jednoznacznego wyznaczenia. Aby zawęzić zbiór rozwiązań do jednego elementu, haker musiałby zdobyć dodatkowe informacje eliminujące $2^{112} - 1$ fałszywych kandydatów.

1. **Przechwycenie nieuszkodzonej ramki z kodem.** Gdyby skaner zarejestrował ramkę bez zniekształceń, kod byłby wprost dostępny. Jest to najprostszy scenariusz: brakujące 128 bitów zostaje dostarczone bezpośrednio.
2. **Przechwycenie dodatkowych bitów kodu z innego źródła.** Dysponując co najmniej 112 dowolnymi bitami kodu (z innej przechwytniętej ramki, przez dostęp fizyczny do urządzenia lub przez atak kanałem bocznym), można wyznaczyć pozostałe 16 bitów z warunku CRC.
3. **Wiedza o strukturze lub formacie kodu.** Jeżeli kod nie jest losowy, lecz ma znany format (stały nagłówek, numer seryjny, ograniczony zakres wartości, generowany przez PRNG o przestrzeni stanu $< 2^{112}$), każde takie ograniczenie zmniejsza liczbę kandydatów. Wystarczające ograniczenie prowadzi do jednoznaczności.

Wniosek. Samo CRC-16 dostarcza tylko 16 bitów informacji o 128-bitowym kodzie. Brakujące 112 bitów haker musi pozyskać z innego źródła: bezpośrednio (kolejna ramka, dostęp do urządzenia) lub pośrednio (wiedza o strukturze kodu).